

Interview Preparation Guide · 3+ Years Experience

15 Questions · 2 Rounds · Detailed Answers with Examples & Code

Round 1	DevOps & CI/CD Assessment	Q1 – Q5
Round 2	Linux, AWS, Jenkins, Docker & Kubernetes	Q6 – Q15

■ Round 1: DevOps & CI/CD Assessment

Q1. What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment? Give a real-world example.

- **Continuous Integration (CI):** Developers frequently merge code into a shared repository; automated builds and tests run on every commit to catch integration bugs early.
- **Continuous Delivery (CD):** Every change that passes CI is automatically built, tested, and made *ready* for production deployment — but a human manually triggers the release.
- **Continuous Deployment:** Every change that passes all automated tests is deployed to production *automatically*, without any manual gate.
- **Real-world example:** A team pushes a feature branch → GitHub Actions runs unit/integration tests (CI) → artifact is built and deployed to staging (Delivery) → after QA sign-off, the release pipeline pushes to prod (Deployment if automated, Delivery if manual).

■ Key interview point: CD can mean both Delivery and Deployment — always clarify which one.

Q2. What is Infrastructure as Code (IaC)? How does Terraform differ from AWS CloudFormation?

- **IaC** is the practice of managing and provisioning infrastructure through machine-readable definition files instead of manual processes — enabling version control, repeatability, and automation.

Dimension	Terraform	AWS CloudFormation
Vendor	HashiCorp (open-source)	AWS-native

Language	HCL (HashiCorp Config Language)	JSON / YAML
Multi-cloud	Yes — AWS, Azure, GCP, etc.	AWS only
State management	Explicit state file (local / S3 / Terraform Cloud)	Managed by AWS internally
Drift detection	terraform plan shows drift	Stack drift detection built-in
Rollback	Manual or with workspaces	Automatic stack rollback on failure
Modules / Reuse	Terraform modules (registry)	Nested stacks / StackSets

■ Use Terraform for multi-cloud or existing Terraform codebases; CloudFormation when you want native AWS integration with no extra tooling.

Q3. Explain the concept of 'shift-left' in DevOps. How does it improve software quality?

- **Shift-left** means moving testing, security, and quality checks *earlier* (to the left) in the SDLC — into development and CI, rather than waiting for QA or production.
- **Key practices:** Static code analysis (SonarQube), unit tests in the IDE, SAST/DAST in the CI pipeline, dependency vulnerability scanning (Snyk, Trivy), and IaC linting (tflint, cfn-lint).
- **Why it improves quality:** Bugs found in development cost ~6× less to fix than in production (IBM Systems Sciences study). Developers get instant feedback and own quality, reducing handoff delays.
- **Example:** A developer commits code → pre-commit hook runs linting → CI pipeline runs SAST → PR blocked if critical vulnerability found — all before the code reaches QA.

■ Shift-left is not about removing QA; it's about making quality everyone's responsibility from day one.

Q4. What is a blue-green deployment strategy? How does it differ from canary deployment?

- **Blue-Green Deployment:** Two identical production environments run simultaneously — Blue (live) and Green (new version). Traffic is switched 100% from Blue to Green after validation. Rollback = flip traffic back.
- **Canary Deployment:** The new version is released to a small subset of users (e.g., 5%) first. Traffic is gradually shifted to 100% if metrics are healthy. Rollback = redirect traffic back to old version.

Dimension	Blue-Green	Canary
Traffic switch	All-at-once (100%)	Gradual (5% → 25% → 100%)
Risk	Binary — works or reverts	Lower — limited blast radius
Cost	Double infrastructure during switch	Minimal extra infra
Rollback speed	Instantaneous (DNS/LB switch)	Fast but gradual
Best for	Predictable releases, DB migrations	Risky changes, A/B testing

■ In AWS: Blue-Green uses Route 53 weighted routing or ALB listener rules; Canary uses CodeDeploy canary config or Kubernetes argo-rollouts.

Q5. What are the key differences between monolithic and microservices architecture from a DevOps perspective?

- **Monolithic:** Single deployable unit. One codebase, one build pipeline, one deployment. Simpler to develop initially but creates bottlenecks — a single bug or slow test suite blocks all teams.
- **Microservices:** Application split into small, independently deployable services. Each team owns its own CI/CD pipeline, deployment cadence, and tech stack.
- **DevOps impact of microservices:** More pipelines to manage; requires service mesh (Istio), API gateways, distributed tracing (Jaeger/X-Ray), and container orchestration (Kubernetes).
- **Deployment complexity:** Monolith = one deployment; Microservices = coordinated multi-service deployments with versioned APIs and backward compatibility requirements.
- **Failure isolation:** In microservices, one service failing doesn't bring down the entire app — but debugging distributed failures is significantly harder.

■ *The DevOps overhead of microservices is often underestimated. You need mature CI/CD, observability, and on-call practices before splitting the monolith.*

■ Round 2: Linux, AWS, Jenkins, Docker & Kubernetes

Q6. What is AWS Elastic Load Balancer (ELB)? Explain the difference between ALB, NLB, and CLB.

- **ELB** distributes incoming traffic across multiple targets (EC2, Lambda, containers) across Availability Zones to ensure fault tolerance and scalability.

Type	Layer	Protocol	Best Use Case
ALB (Application LB)	Layer 7	HTTP / HTTPS / WebSocket	Microservices, path/host-based routing, gRPC
NLB (Network LB)	Layer 4	TCP / UDP / TLS	Ultra-low latency, static IPs, gaming, IoT
CLB (Classic LB)	Layer 4 & 7	HTTP / HTTPS / TCP	Legacy — avoid for new architectures

■ *ALB is the default choice for web applications. NLB is needed when you require static Elastic IPs or sub-millisecond latency.*

Q7. What is AWS CloudWatch? How do you set up alarms and custom metrics for your application?

- **CloudWatch** is AWS's unified observability service providing metrics, logs, alarms, dashboards, and events across AWS resources and custom applications.
- **Setting up alarms:** Navigate to CloudWatch → Alarms → Create Alarm → select a metric (e.g., EC2 CPUUtilization) → define threshold → configure SNS notification or Auto Scaling action.
- **Custom metrics** are published via the AWS SDK or CLI:

```
aws cloudwatch put-metric-data \ --namespace "MyApp/Performance" \ --metric-name
"OrderProcessingTime" \ --value 350 \ --unit Milliseconds \ --dimensions
Service=OrderService,Env=prod
```

- **Log Insights:** Use CloudWatch Logs Insights to query application logs with SQL-like syntax for debugging and performance analysis.

■ Always use metric filters to extract numeric signals from log data — e.g., counting `ERROR` log lines and alarming when they exceed a threshold.

Q8. Explain the concept of AWS Lambda. What are cold starts and how do you minimize them?

- **AWS Lambda** is a serverless compute service that runs code in response to events (HTTP via API Gateway, S3 events, SQS messages, etc.) without provisioning or managing servers. You pay only for execution time.
- **Cold start:** When Lambda receives a request and no warm execution environment exists, AWS must initialize a new container — downloading the code package, starting the runtime, and running the init code. This adds 100ms–1s+ latency.
- **Minimizing cold starts:**
 - — **Provisioned Concurrency:** Pre-warms a specified number of execution environments so they're always ready.
 - — **Reduce package size:** Smaller deployment packages initialize faster. Use Lambda Layers for shared dependencies.
 - — **Choose runtime wisely:** Compiled runtimes (Go, Rust) have significantly faster cold starts than JVM (Java/Kotlin).
 - — **Avoid VPC Lambda unless necessary:** VPC attachment adds ENI creation latency, though AWS has improved this with hyperplane ENIs.
 - — **Keep Lambda warm:** For critical functions, use EventBridge scheduled rules to ping the function every 5 minutes.

■ *Provisioned Concurrency is the production-grade solution; warming via pings is a workaround that becomes unreliable at scale.*

Q9. What is the difference between Security Groups and Network ACLs in AWS?

Dimension	Security Groups	Network ACLs
Level	Instance / ENI level	Subnet level
State	Stateful (return traffic auto-allowed)	Stateless (must explicitly allow inbound + outbound)
Rules	Allow rules only	Allow and Deny rules
Rule evaluation	All rules evaluated together	Rules evaluated in order (lowest number first)
Default behavior	Deny all inbound, allow all outbound	Allow all inbound and outbound
Scope	Applies to specific EC2 instances	Applies to all instances in the subnet

■ Use NACLs as a subnet-level firewall for broad blocking (e.g., blocking a malicious IP range) and Security Groups for fine-grained instance control.

Q10. How does Jenkins handle parallel execution of stages in a pipeline? Write an example snippet.

- Jenkins Declarative Pipeline supports a **parallel** block within a stage, allowing multiple branches to run simultaneously on available agents.
- Use cases: running unit tests, integration tests, and lint checks concurrently to reduce total pipeline duration.

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'mvn package -DskipTests'
            }
        }
        stage('Parallel Tests') {
            parallel {
                stage('Unit Tests') {
                    steps {
                        sh 'mvn test -Dgroups=unit'
                    }
                }
                stage('Integration Tests') {
                    steps {
                        sh 'mvn test -Dgroups=integration'
                    }
                }
                stage('Code Quality') {
                    steps {
                        sh 'mvn sonar:sonar'
                    }
                }
            }
        }
    }
}
```

■ Add 'failFast: true' inside the parallel block to abort all branches immediately if one fails — critical for fast feedback.

Q11. What are shared libraries in Jenkins and why are they used in large-scale CI/CD setups?

- **Jenkins Shared Libraries** are reusable Groovy code stored in a separate Git repository and loaded into multiple pipelines via `@Library` annotation.
- **Why they exist:** In large organizations, dozens of teams write nearly identical CI/CD logic (build, test, push image, deploy). Shared libraries centralize this logic so changes propagate everywhere at once.
- **Structure:** Repo contains **vars/** (global functions callable directly in Pipelines), **src/** (Groovy classes), and **resources/** (config files, scripts).
- **Usage example:**

```
// Jenkinsfile @Library('my-shared-lib@main') _ pipeline { agent any stages { stage('Build & Push') { steps { // Calling a shared function from vars/dockerBuildPush.groovy dockerBuildPush(imageName: 'my-app', tag: env.BUILD_NUMBER) } } } }
```

■ Version-pin shared libraries (e.g., `@Library('lib@v1.2.3')`) in production pipelines to avoid unexpected breakage from library updates.

Q12. What is the difference between Docker COPY and ADD instructions? When would you use each?

- **COPY** copies files or directories from the build context into the image. Simple, predictable, and the recommended default.
- **ADD** does everything COPY does, *plus*: automatically extracts tar archives (e.g., `.tar.gz`) and can fetch files from remote URLs.
- **When to use COPY:** Almost always. Use COPY for all file transfers unless you specifically need tar extraction or URL fetching.
- **When to use ADD:** Only when you need automatic tar extraction (e.g., `ADD app.tar.gz /app/`). Avoid using ADD for URLs — `wget/curl` is more explicit and cacheable.
- **Best practice:** The official Docker documentation recommends preferring COPY for transparency. ADD's implicit behaviors (auto-extract, remote URL) can introduce surprises in CI/CD.

```
# Preferred - clear and predictable COPY src/ /app/src/ COPY requirements.txt /app/ # Use ADD only for tar extraction ADD dist.tar.gz /opt/app/
```

Q13. Explain Docker multi-stage builds. How do they help reduce image size in production?

- **Multi-stage builds** use multiple FROM instructions in a single Dockerfile. Each stage can copy artifacts from previous stages, discarding build tools and intermediate files.
- **Problem solved:** A Java/Go application needs a full JDK/compiler to build but only needs the JRE/binary at runtime. Without multi-stage builds, the production image carries unnecessary build tooling (~500MB+).

```
# Stage 1: Builder FROM maven:3.9-eclipse-temurin-21 AS builder WORKDIR /app COPY pom.xml . RUN mvn dependency:go-offline COPY src ./src RUN mvn package -DskipTests # Stage 2: Production image (only the JRE) FROM eclipse-temurin:21-jre-alpine WORKDIR /app # Only the JAR is copied — no Maven, no source code COPY --from=builder /app/target/app.jar ./app.jar EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"]
```

- **Size impact:** Builder stage ~500MB → Production image ~120MB (JRE Alpine). No source code, no compiler, no test dependencies in the final image.

■ Multi-stage builds also improve security — smaller attack surface means fewer CVEs to patch in the runtime image.

Q14. What is a Kubernetes ConfigMap and Secret? How do they differ and how are they mounted into pods?

- **ConfigMap:** Stores non-sensitive configuration data as key-value pairs (e.g., app settings, feature flags, environment names). Data is stored in plaintext in etcd.
- **Secret:** Stores sensitive data (passwords, API keys, TLS certs). Values are base64-encoded in the manifest but can be encrypted at rest in etcd with KMS.

Dimension	ConfigMap	Secret
Data type	Non-sensitive config	Sensitive credentials, certs
Storage	Plaintext in etcd	Base64-encoded (encrypt at rest with KMS)
Size limit	1 MiB	1 MiB
Access in pod	Env var or volume mount	Env var or volume mount (tmpfs)

- **Mounting as volume (recommended for Secrets):**

```
volumes: - name: db-secret secret: secretName: db-credentials containers: - name: app
volumeMounts: - name: db-secret mountPath: /etc/secrets readOnly: true env: - name: APP_ENV #
ConfigMap as env var valueFrom: configMapKeyRef: name: app-config key: environment
```

■ Prefer volume mounts over env vars for Secrets — env vars are visible via 'kubectl describe pod' and can leak into logs. Use AWS Secrets Manager with External Secrets Operator for production.

Q15. How do you write a Linux shell script to monitor a service and restart it automatically if it goes down?

- The script checks if the service process is running using **systemctl** or **pgrep**, logs the status, and restarts if the service is inactive.

```
#!/bin/bash # monitor_service.sh – Monitor and auto-restart a systemd service
SERVICE_NAME="nginx" LOG_FILE="/var/log/service_monitor.log" TIMESTAMP=$(date '+%Y-%m-%d
%H:%M:%S') # Function: log to file and stdout log() { echo "[$TIMESTAMP] $1" | tee -a
"$LOG_FILE" } # Check service status if ! systemctl is-active --quiet "$SERVICE_NAME"; then
log "WARNING: $SERVICE_NAME is DOWN. Attempting restart..." systemctl restart "$SERVICE_NAME"
# Verify restart succeeded if systemctl is-active --quiet "$SERVICE_NAME"; then log "SUCCESS:
$SERVICE_NAME restarted successfully." else log "ERROR: Failed to restart $SERVICE_NAME.
Manual intervention required." # Optional: send alert via SNS or email # aws sns publish
--topic-arn arn:aws:sns:... --message "Service $SERVICE_NAME down!" fi else log "OK:
$SERVICE_NAME is running." fi
```

- **Schedule via cron** (every 2 minutes):

```
# Add to crontab: crontab -e */2 * * * * /usr/local/bin/monitor_service.sh
```

- **Better alternative:** For systemd services, configure automatic restart natively — more reliable than cron:

```
# /etc/systemd/system/nginx.service [Service] Restart=always RestartSec=5s
```

■ Systemd's Restart=always is the production-grade approach. The shell script is useful for non-systemd processes or when you need custom alerting logic.

This guide covers 15 questions across 2 interview rounds for an AWS DevOps Engineer role (3+ years experience).

Topics: CI/CD · IaC · Shift-Left · Deployment Strategies · AWS ELB · CloudWatch · Lambda · Security · Jenkins · Docker · Kubernetes · Linux Scripting